What is System Software ?

Clemens Grelck



System Software

What is System Software ?

[Placeholder for a very nice graphic]

In a short term:

Anything in between

- **computing hardware**

and

- **application software.**

What is System Software ?

[Placeholder for a very nice graphic]

In a short term:

Anything in between

- **computing hardware**

and

- **application software.**

Application software:

- used by humans
- directly or indirectly

System software:

- used by application software
- directly or indirectly

Operating System

Purpose:

- makes hardware accessible to software
- lowest level of (system) software stack

Properties:

- virtualises the processor
- virtualises the memory
- virtualises persistent memory (hard disks, solid state disks, ...)
- virtualises the network interface
- virtualises ...
- runs itself in privileged mode
- runs everything else in non-privileged mode

Programming Languages

Purpose:

- Who *speaks* binary other than computers?
- Programmers view of computers
- Abstract from concrete hardware and low-level details
- Achieve portabiliy
- Allow us to express algorithmic concepts

Programming Languages

Purpose:

- Who *speaks* binary other than computers?
- Programmers view of computers
- Abstract from concrete hardware and low-level details
- Achieve portabiliy
- Allow us to express algorithmic concepts

Huge diversity in designs and features:

- 100s of more widely used languages
- 100,000s of *research* languages
- On-going area of research
- New languges proposed on a daily basis
- Are the ones we use really the best ?
- Trend: domain specific languages (DSL)

Programming Language Paradigms:
Classes of languages with similar fundamental properties

Programming Language Paradigms: Classes of languages with similar fundamental properties

Imperative:

- Statements step-wise manipulate state
- Global and local variables
- (Structured control flow: loops, etc)
- Static binding
- Based on Turing-machine
- C, Fortran, Ada, Cobol, Rust, Julia, ...

Programming Language Paradigms: Classes of languages with similar fundamental properties

Imperative:

- Statements step-wise manipulate state
- Global and local variables
- (Structured control flow: loops, etc)
- Static binding
- Based on Turing-machine
- C, Fortran, Ada, Cobol, Rust, Julia, ...

Object-oriented:

- Likewise imperative
- State (mostly) encapsulated in objects
- Subtyping via class hierarchies
- Dynamic / late binding
- Smalltalk, C++, Java, Scala, C#, ...

Programming Language Paradigms: Classes of languages with similar fundamental properties

Imperative:

- Statements step-wise manipulate state
- Global and local variables
- (Structured control flow: loops, etc)
- Static binding
- Based on Turing-machine
- C, Fortran, Ada, Cobol, Rust, Julia, ...

Object-oriented:

- Likewise imperative
- State (mostly) encapsulated in objects
- Subtyping via class hierarchies
- Dynamic / late binding
- Smalltalk, C++, Java, Scala, C#, ...

Functional:

- Programming by recursive equations
- Functions as first-class citizens
- No state
- Based on λ-calculus
- Lisp, Scheme, SML, OCaml, Haskell, ...

Programming Language Paradigms: Classes of languages with similar fundamental properties

Imperative:

- Statements step-wise manipulate state
- Global and local variables
- (Structured control flow: loops, etc)
- Static binding
- Based on Turing-machine
- C, Fortran, Ada, Cobol, Rust, Julia, ...

Object-oriented:

- Likewise imperative
- State (mostly) encapsulated in objects
- Subtyping via class hierarchies
- Dynamic / late binding
- Smalltalk, C++, Java, Scala, C#, ...

Functional:

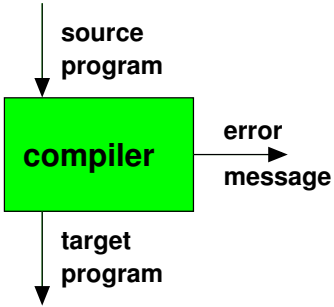
- Programming by recursive equations
- Functions as first-class citizens
- No state
- Based on λ-calculus
- Lisp, Scheme, SML, OCaml, Haskell, ...

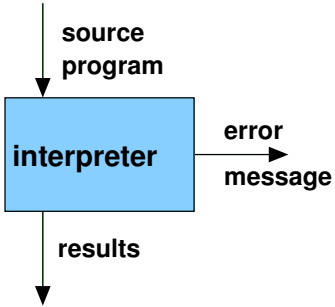
Logic:

- Free variables
- Non-deterministic choices
- Built-in search facility to find variable bindings that satisfy constraints
- Prolog, Curry, ...

Compilers and Interpreters

Comparison:





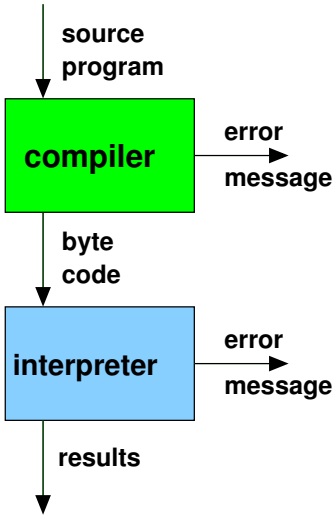
Similarities:

- Analysis of source code
- Optimisation

Differences:

- Interpreter *executes* code.
- Compiler *describes* execution.

Hybrid Architectures: Compiler and Interpreter



- Compiler produces intermediate code for **virtual machine**
- Interpreter implements virtual machine on real hardware
- Popular approach since the advent of Java
- Advantage:
  code portability and code motion
- Disadvantage:
  less efficient than native code

Runtime Systems / Runtime Environments

Dynamic services expected by compiled code:

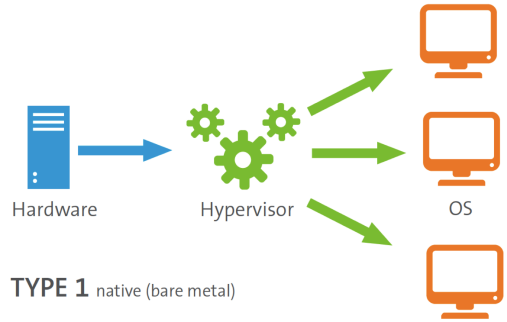
- On top of operating system
- Non-privileged code
- Specific to programming language and compiler

Examples:

- Memory allocators
- Garbage collectors
- Threading environments

Hypervisors

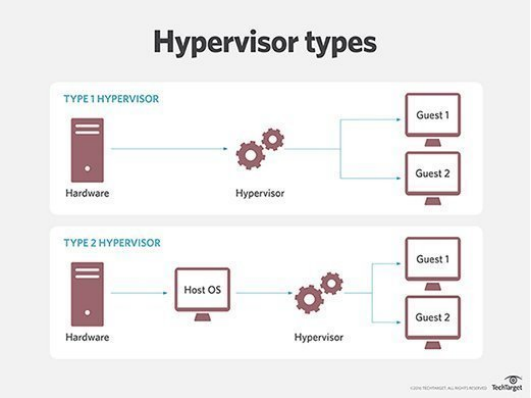
Meta virtualisation:



Idea:

- Hypervisor provides essential (bare-metal) OS services
- Virtualises the entire machine (*virtual machine*)
- VM runs fully-fledged guest OS
- Application only experiences the regular OS
- Popular with cloud computing
- Examples: VMware, VirtualBox, ...

Types of Hypervisors



Type 1 — bare metal:

- Hypervisor as light-weight OS
- Runs fully-fledged operating systems
- Example: WMware

Type 2 — hosted:

- Regular OS
- Hypervisor as (non-privileged) software layer
- Runs alternative OS
- Example: Windows Subsystem for Linux (WSL)

Conclusion

System Software:

- Rich and diverse area of computer science
- Too much for one course

Conclusion

System Software:

- Rich and diverse area of computer science
- Too much for one course

System Software course at FSU:

- Legacy course on operating systems
- Just one aspect of the world of system software
- But then in-depth